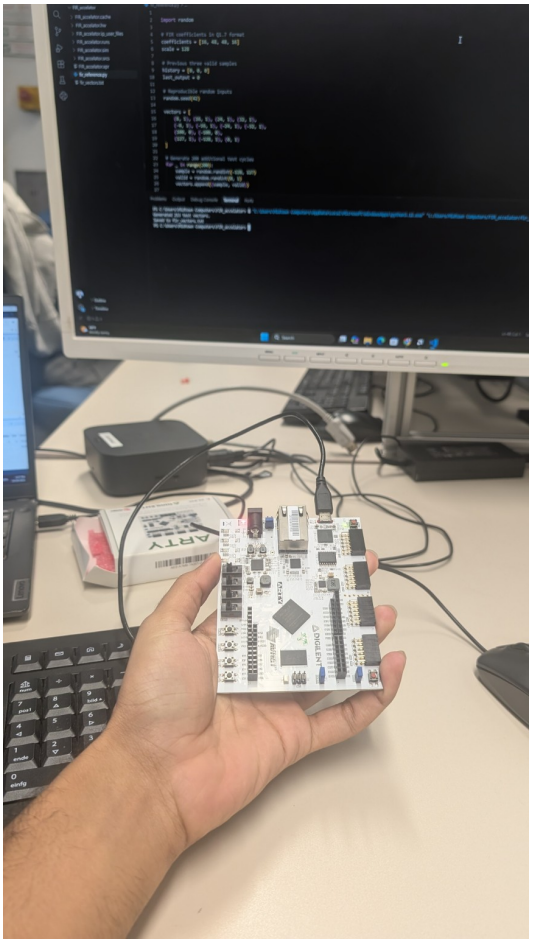
FPGA FIR ACCELERATOR

Complete design, verification, synthesis, timing, and physical hardware demonstration report

Target: Digilent Arty Artix-7 35T Rev C | FPGA: xc7a35ticsg324-1L | Tool: Vivado 2026.1 | Host: Windows / VS Code / Python 3.12 | Date: 28 October 2026

Scope: A 4-tap, signed 8-bit, Q1.7 programmable-coefficient FIR filter with three registered arithmetic stages, UART interface at 115200 baud, Python reference verification, and a hardware-in-the-loop dashboard.

Evidence policy: Reported performance and pass results come from screenshots and textual simulator logs supplied in this conversation. Explanatory architecture descriptions and worked calculations are distinct from measured timing. No unseen waveforms, RTL files, test logs, or on-board throughput measurements are claimed.



Physical Digilent Arty board connected by USB during the project | Original: 168250.jpg

Executive summary

The project evolved from a four-sample moving average into a three-stage pipelined, runtime-programmable FIR accelerator. In simulation, Python-generated vectors checked 213 sample/valid cycles against baseline and pipelined designs: 432 comparisons passed. A separate programmable-FIR simulation passed 16 checked cycles. A hardware UART loopback passed, followed by eight signed FIR hardware examples, eight coefficient-reprogramming examples after a manual reset, and an interactive dashboard showing 13 matched physical FPGA samples with zero mismatches.

The programmable FIR implementation occupied 153 LUTs, 96 registers, 59 slices and 2 DSP48E1s in the earlier standalone post-route report, with +3.402 ns setup WNS at a 10 ns clock target. Later UART hardware integration is a different implemented design; do not conflate its utilization with the standalone FIR measurements.

1. Requirements and system overview

Item	Implemented specification
Board	Digilent Arty Artix-7 35T Rev C; xc7a35ticsg324-1L
Host tools	Vivado 2026.1, Windows, VS Code, Python, PySerial, Matplotlib
System clock	100 MHz; 10.000 ns clock constraint
Interface	One USB cable: board power + JTAG configuration + virtual COM5 USB-UART
UART format	115200 baud, 8 data bits, no parity, 1 stop bit
Filter	4 taps, signed 8-bit samples, signed 8-bit coefficients (Q1.7)
Commands	S + one sample byte; C + four coefficient bytes; K ACK for coefficient loading
Status	Physical UART and FIR processing verified; arbitrary coefficient saturation and continuous streaming not implemented

System architecture

Windows Python -> USB-UART COM5 -> FPGA UART RX -> command decoder -> three-stage programmable FIR -> FPGA UART TX -> Python verification + charts. The JTAG configuration path shares the physical cable but is logically separate from UART data.

The controller sends one sample and waits for its result. This validates the processing pipeline but does not demonstrate sustained one-sample-per-clock external data streaming; at 115200 baud the serial protocol substantially limits end-to-end throughput.

2. FIR theory and numerical representation

General N-tap FIR: **y[n] = sum from k=0 to N-1 of h[k] x[n-k]**. For four taps with signed Q1.7 integer coefficients **H[k]**, this project computes **y[n] = floor((H0*x[n] + H1*x[n-1] + H2*x[n-2] + H3*x[n-3]) / 128)**. In RTL, an arithmetic right shift by seven bits implements this scaling; for negative nonmultiples of 128 it rounds toward negative infinity. The 8-bit output can wrap if the result exceeds -128..127.

Property	Value / consequence
Sample width	Signed 8-bit: -128 to +127
Coefficient width	Signed 8-bit integer Q1.7: -128 to +127, interpreted as value/128
Per-tap product	8-bit x 8-bit -> full 16-bit signed product

Property	Value / consequence
Pair sums	17-bit signed
Final accumulator	18-bit signed
Output	Arithmetic shift right 7; cast to signed 8-bit (wrap on overflow)
Coefficients, smoothing	[16,48,48,16] / 128 = [0.125,0.375,0.375,0.125]
Coefficients, average	[32,32,32,32] / 128 = four weights of 0.25

Worked arithmetic example — default coefficients

With newest to oldest samples [32, 24, 16, 8] and coefficients [16,48,48,16]:

```
(16*32 + 48*24 + 48*16 + 16*8) / 128
```

```
= (512 + 1152 + 768 + 128) / 128 = 2560 / 128 = 20
```

This is exactly the reported hardware test result for input 32 after valid history [24,16,8].

Worked arithmetic example — programmable average

With newest-to-oldest samples [40,30,20,10] and coefficients [32,32,32,32], the result is (1280+960+640+320)/128 = **25**. Equal coefficients produce a four-sample moving average.

Sample history behavior

After each valid sample the shift registers retain the previous three inputs. A coefficient change updates the coefficient registers only; it does not clear input history. Pressing BTN0 resets history and restores default coefficients. A failed first hardware coefficient test was resolved by pressing BTN0, confirming the need to synchronize the Python reference state with hardware state.

3. Architecture evolution

Stage A — unsigned moving-average proof of concept

Four history registers x0..x3 were initialized to zero. The first baseline computed the previous four values (one accepted-sample offset), then was corrected to include the current sample plus three stored values. A 10-bit accumulator handled four unsigned 8-bit values since 4*255 = 1020, requiring ten bits. The basic testbench passed six corrected cases.

Stage B — signed fixed-coefficient FIR

Coefficients [16,48,48,16] converted the simple average into weighted convolution. Products were represented in 16 bits and accumulated with 18-bit headroom. A manually calculated testbench initially contained three wrong expected values; independently recomputed values fixed those errors.

Stage C — three-stage registered pipeline

Stage 1 registers four products; stage 2 registers two partial sums; stage 3 computes and registers the scaled output. A sample captured at rising edge 1 becomes visible after rising edge 3, equivalent to two additional clock periods compared with a one-register design. Pipeline latency is not throughput: it can accept samples on consecutive edges when sample_valid is asserted. Valid bits are pipelined alongside data.

Stage D — programmable coefficients

The four 8-bit coefficient inputs are loaded into registers on coeff_load. The first sample accepted on the same rising edge as coeff_load uses the old coefficient values, because nonblocking updates occur together; subsequent accepted samples use the newly stored coefficients. The implementation changed resource mapping, including two observed DSP48E1 blocks.

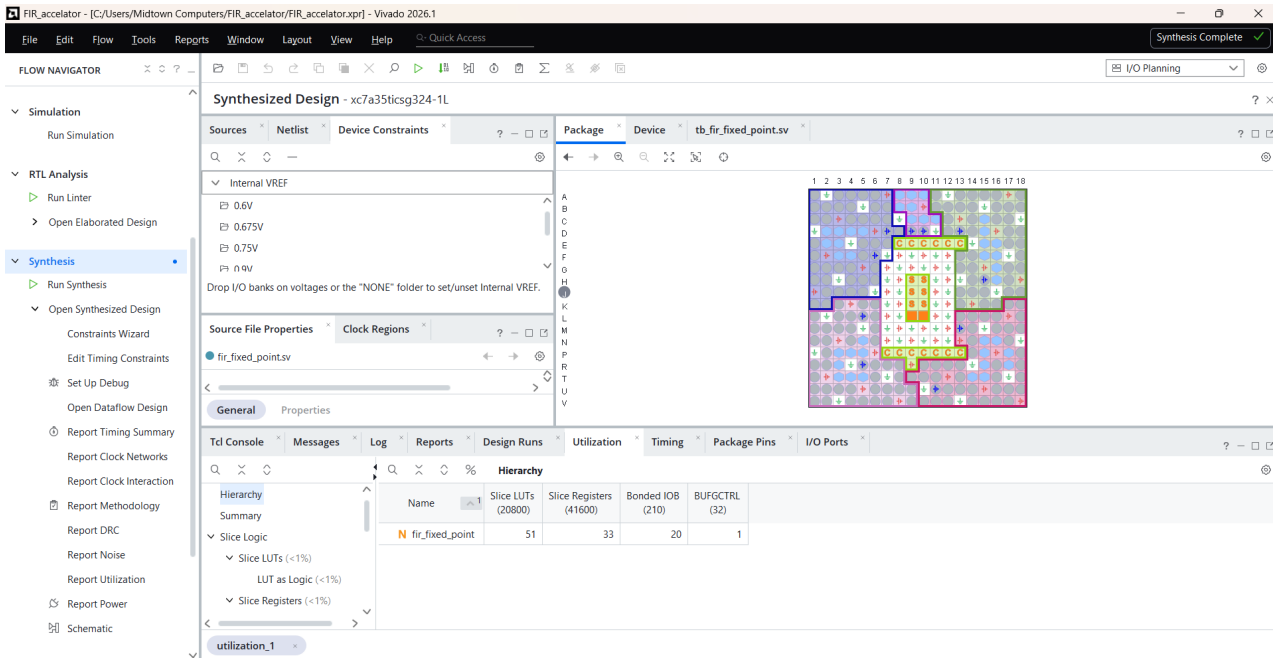
4. Synthesis and timing comparison

The following values are directly transcribed from the supplied Vivado synthesis screenshots; all are from the same Artix-7 target and 10 ns clock goal. **WNS is setup timing margin**, not elapsed processing time.

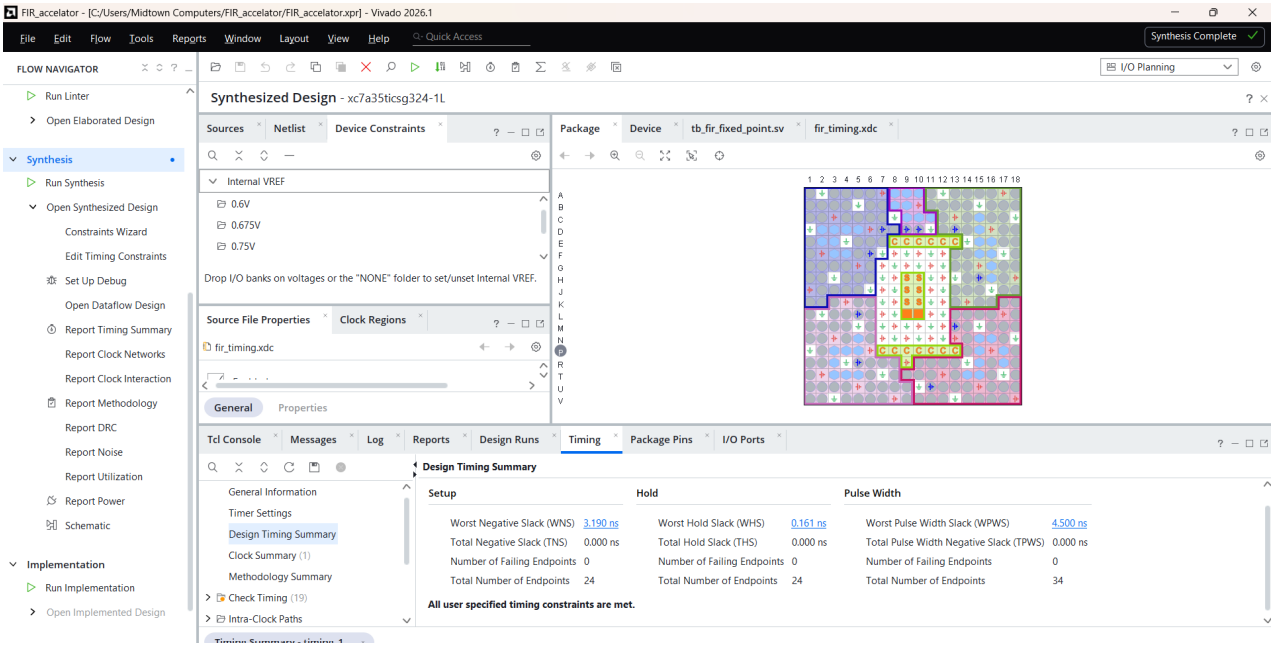
Metric	Fixed, nonpipeline	Fixed, pipeline	Programmable pipeline
LUTs	51	58	153
Flip-flops	33	87	96
DSP48E1	Not individually confirmed	Not individually confirmed	2 observed
Bonded I/O	20	20	53
Setup WNS, ns	+3.190	+5.576	+4.230
Timing stage	Post-synthesis	Post-synthesis	Post-synthesis

Pipeline impact: setup slack improved by 5.576 - 3.190 = **2.386 ns** (+74.8% relative to baseline WNS); LUTs increased by **7** (51->58, +13.7%); FFs increased by **54** (33->87, +163.6%). This is the expected latency/register tradeoff. It does not establish a post-route fMAX increase because baseline and pipeline were not both implemented and measured on the same routed basis.

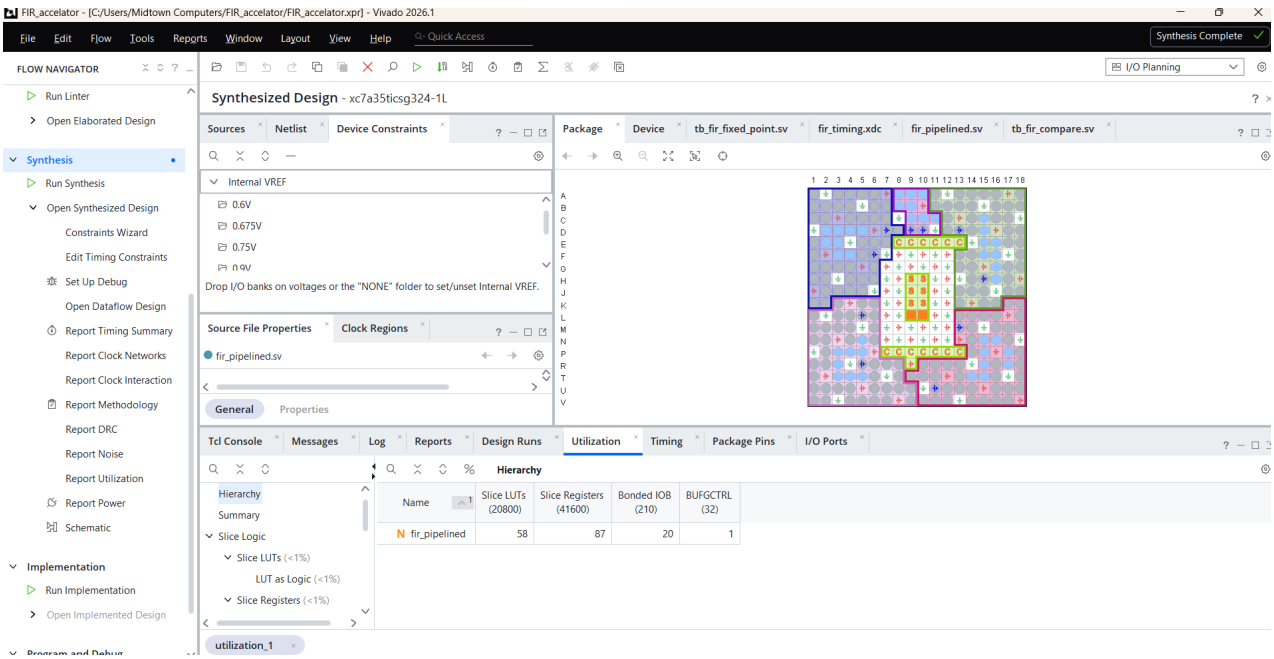
The programmable design additionally permits coefficient changes without recompiling. The 2 DSP48E1 allocation is a synthesis/implementation observation, not evidence that all four multiplications use DSPs: constant and variable multiplier mapping is optimizer dependent.



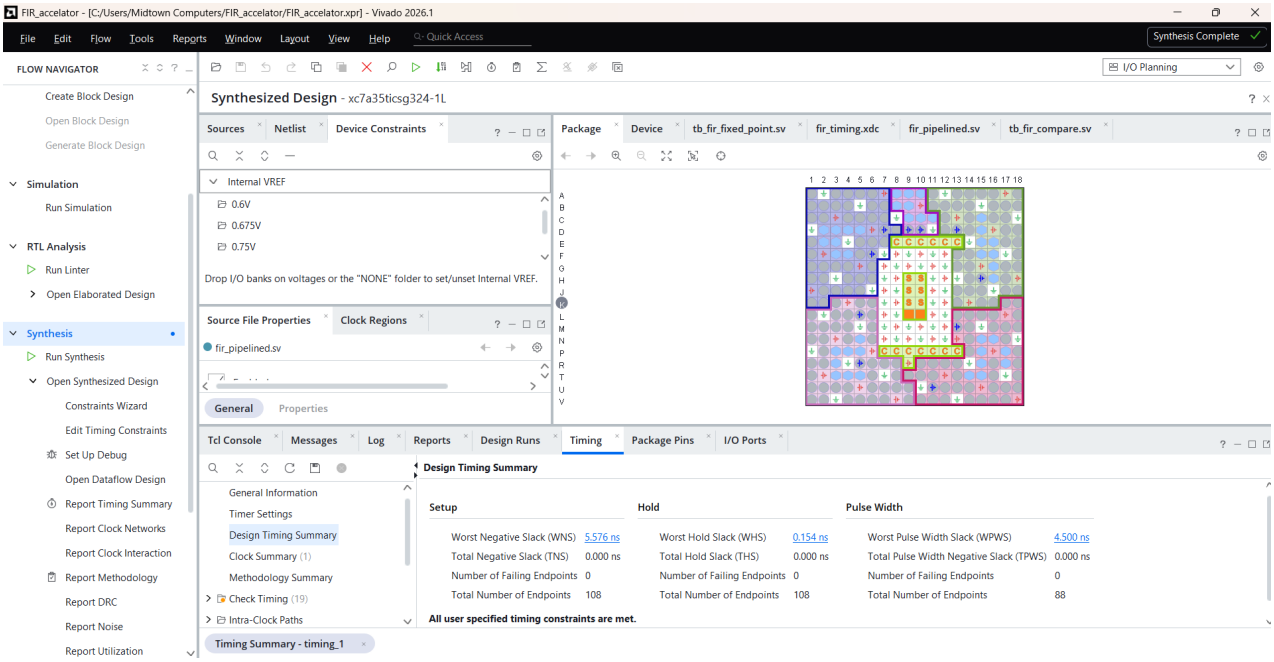
Evidence: original fixed FIR synthesized resources (51 LUTs / 33 FFs). | Original: image(20261028-141700).png



Evidence: fixed FIR post-synthesis setup WNS +3.190 ns. | Original: image(20261028-143311).png



Evidence: pipelined FIR synthesis resources (58 LUTs / 87 FFs). | Original: image(20261028-144249).png

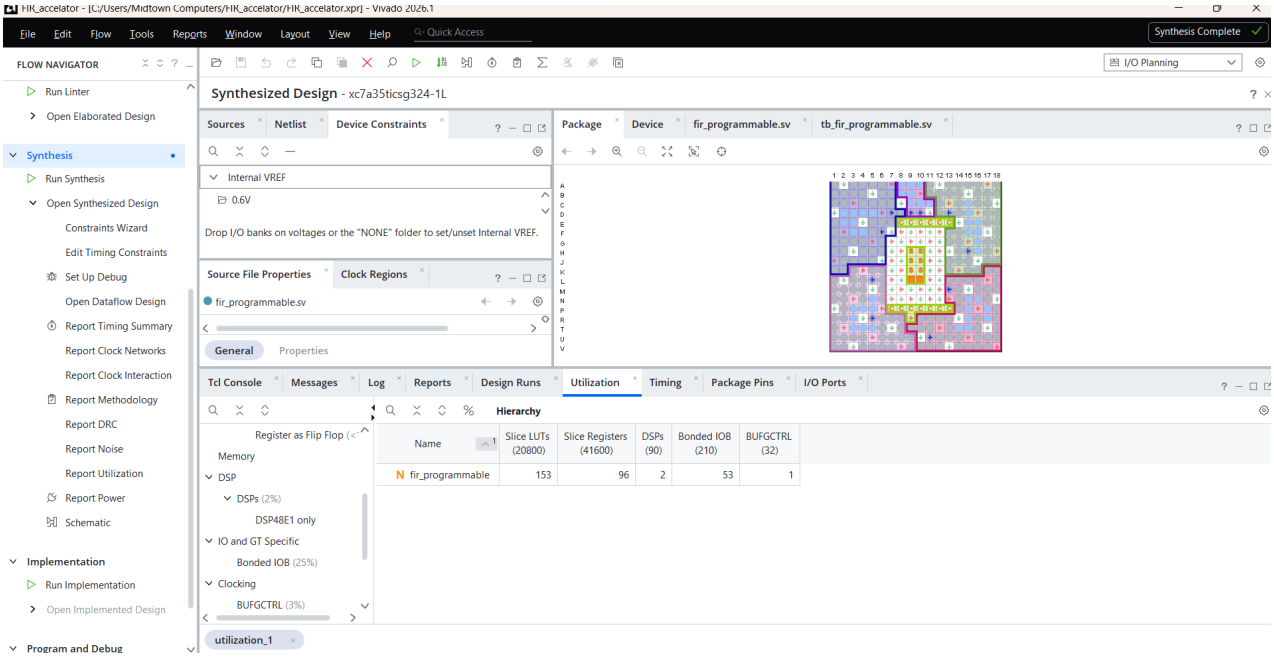


Evidence: pipelined FIR setup WNS +5.576 ns. | Original: image(20261028-144232).png

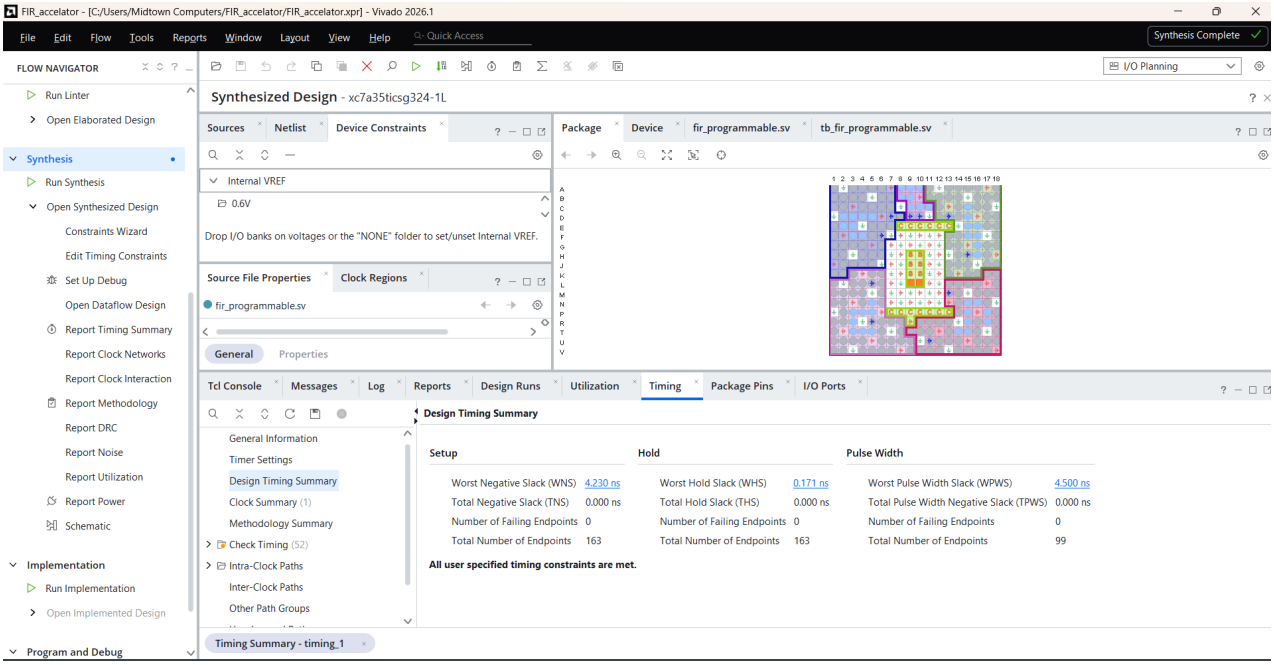
5. Programmable FIR implementation measurements

Metric	Standalone programmable FIR
Synthesis WNS	+4.230 ns
Post-route implementation WNS	+3.402 ns
Post-route hold WHS	+0.221 ns
Failing setup / hold endpoints	0 / 0
LUTs / FFs / Slices	153 / 96 / 59
DSP48E1 blocks	2 out of 90
Bonded I/O / BUFGCTRL	53 / 1

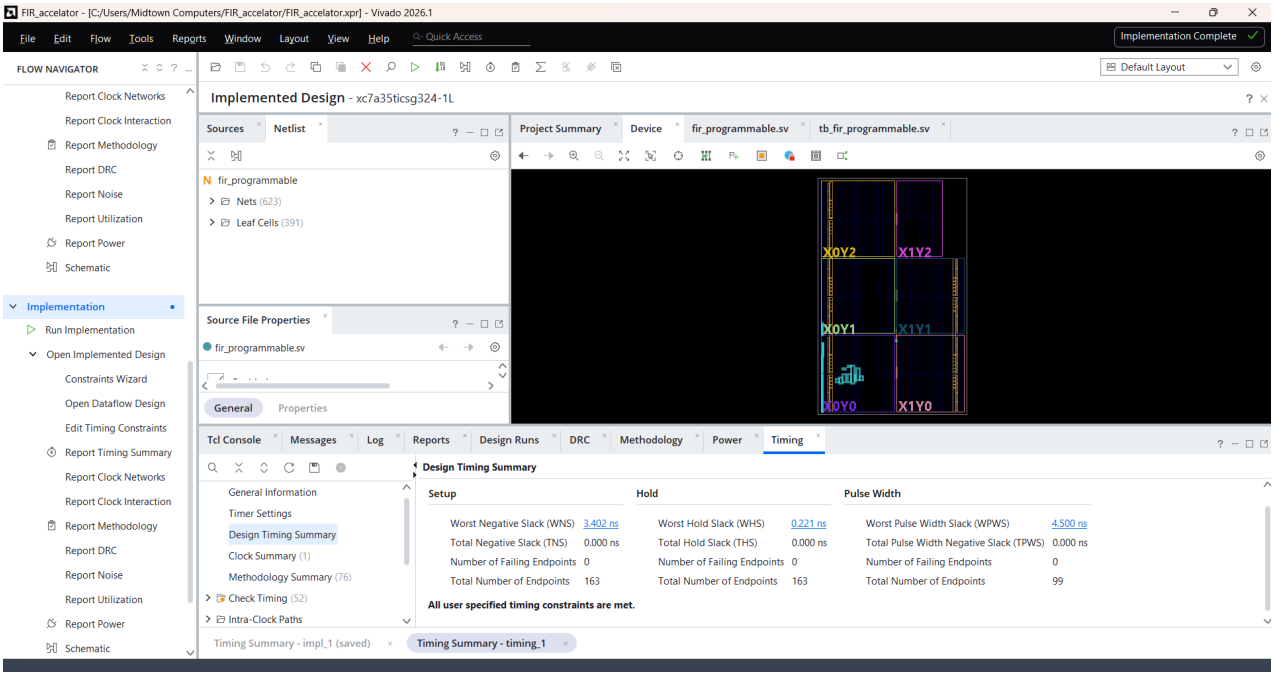
Physical routing reduced setup margin by 4.230 - 3.402 = **0.828 ns**. The 10 ns constrained internal paths passed. Since external I/O delays were not comprehensively defined, the reports should not be presented as complete board-level timing guarantees.



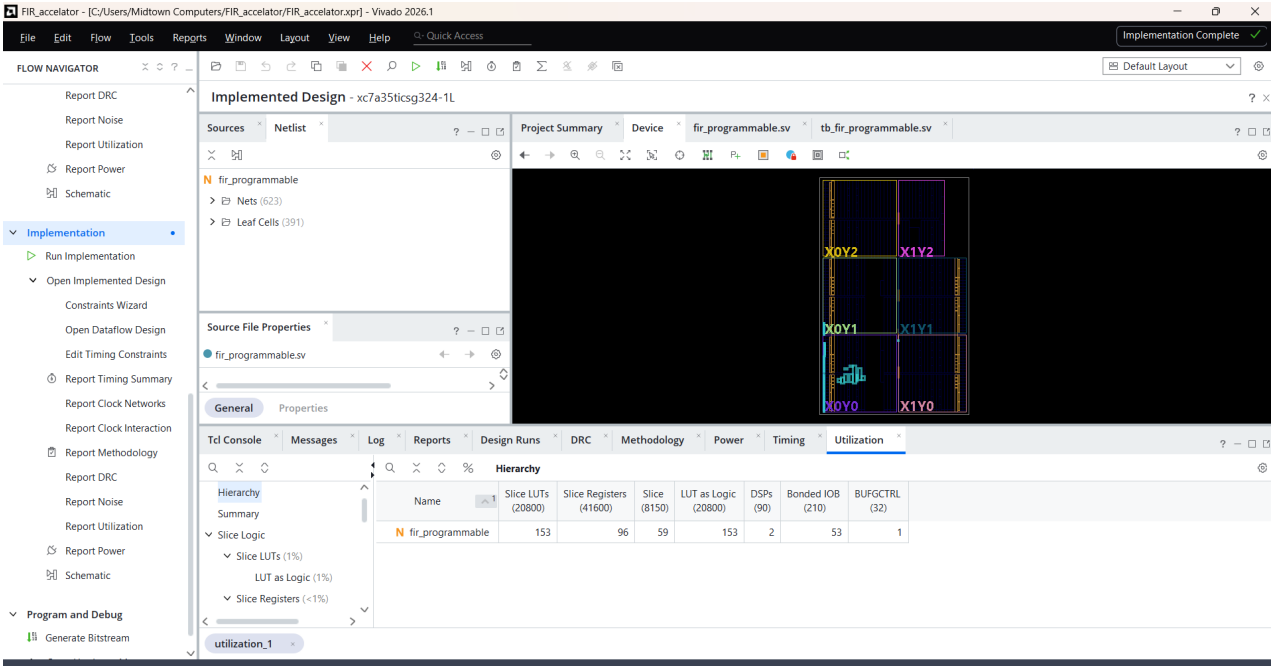
Evidence: programmable FIR synthesis includes two DSP48E1s. | Original: image(20261028-153021).png



Evidence: programmable FIR post-synthesis timing WNS +4.230 ns. | Original: image(20261028-153231).png



Evidence: programmable FIR post-route timing WNS +3.402 ns. | Original: image(20261028-153503).png



Evidence: programmable FIR post-route utilization including 2 DSP48E1s. | Original: image(20261028-153522).png

6. Verification methodology and actual results

Verification campaign	Observed outcome
Moving-average self-check	6 PASS examples
Fixed signed FIR manually corrected test	9 PASS checks
Independent fixed vs pipelined comparison	20 comparison checks PASS
Python golden model + both RTLs	213 input vectors; 432 checks PASS
Programmable coefficient RTL test	16 cycles PASS

Verification campaign	Observed outcome
UART RX/TX loopback simulation	A (65) received back; PASS
Physical UART loopback, COM5	Sent A / received A; PASS
Physical default FIR	8 inputs and 8 matching expected outputs
Physical changed-coefficient FIR	8 inputs and 8 matching expected outputs after BTN0
Dashboard physical run	13 samples / 13 matches / 0 mismatches

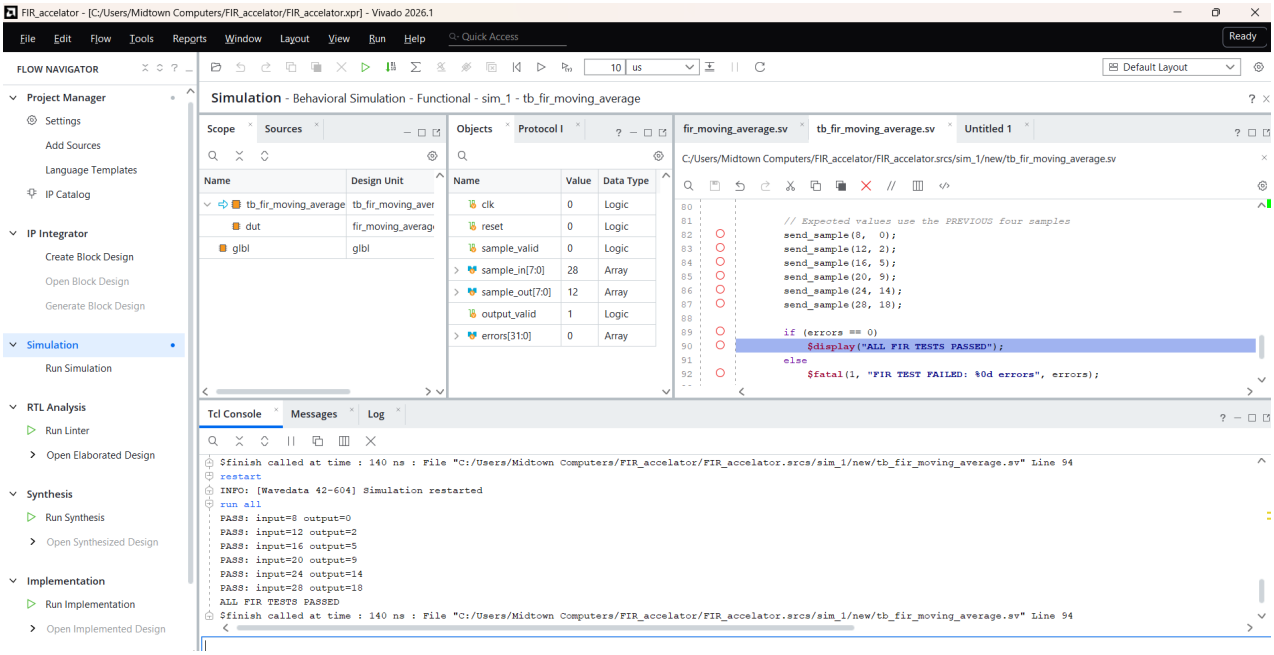
A Python script generated deterministic signed test inputs, sample_valid flags and golden outputs from the same mathematical FIR definition but using independent Python arithmetic. The simulation testbench read the vector file and compared the RTL results with pipeline-aligned expected outputs. These are test-coverage claims for executed vectors, not proof of exhaustive correctness.

Default-coefficient physical result sequence

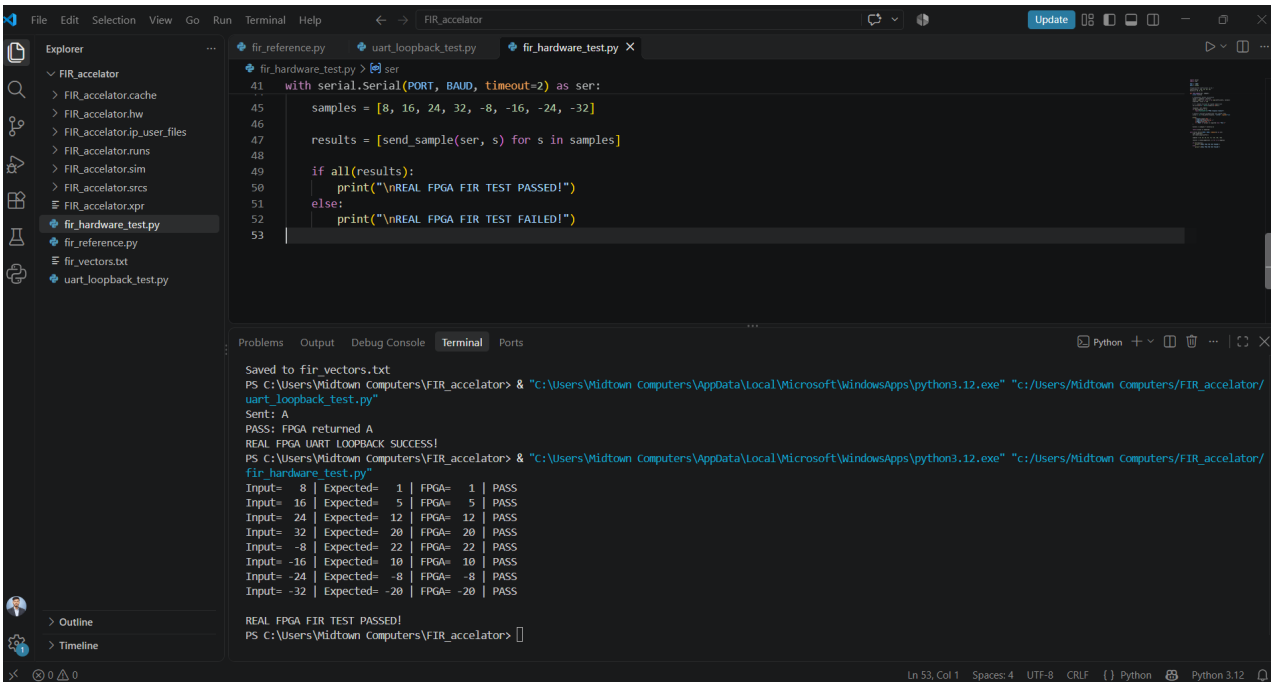
Input	Python expected	FPGA actual	Result
8	1	1	PASS
16	5	5	PASS
24	12	12	PASS
32	20	20	PASS
-8	22	22	PASS
-16	10	10	PASS
-24	-8	-8	PASS
-32	-20	-20	PASS

Equal-weight coefficient physical sequence after BTN0

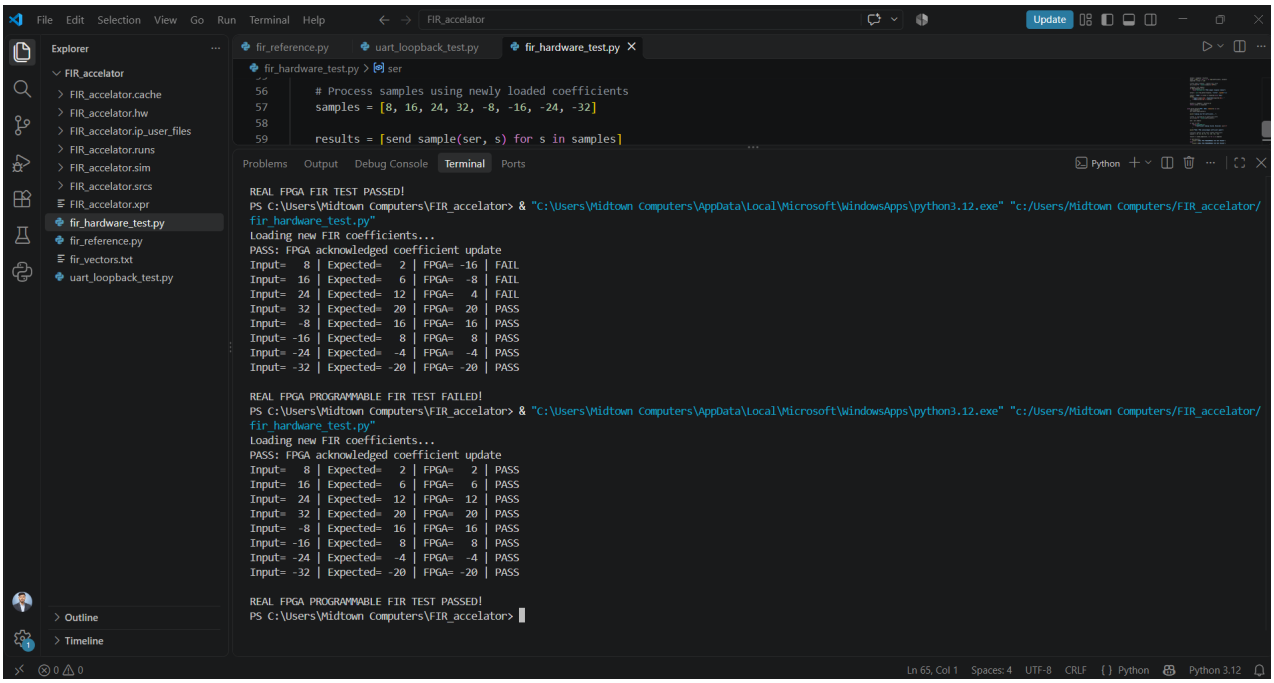
Input	Python expected	FPGA actual	Result
8	2	2	PASS
16	6	6	PASS
24	12	12	PASS
32	20	20	PASS
-8	16	16	PASS
-16	8	8	PASS
-24	-4	-4	PASS
-32	-20	-20	PASS



Evidence: six moving-average RTL checks passed. | Original: image(20261028-133443).png



Evidence: eight physical signed FIR examples matched Python on COM5. | Original: image(20261028-162930).png



Evidence: coefficient update tests before and after the BTN0 reset; second run passed 8/8. | Original: image(20261028-163234).png

7. UART, pin constraints, bitstream and hardware

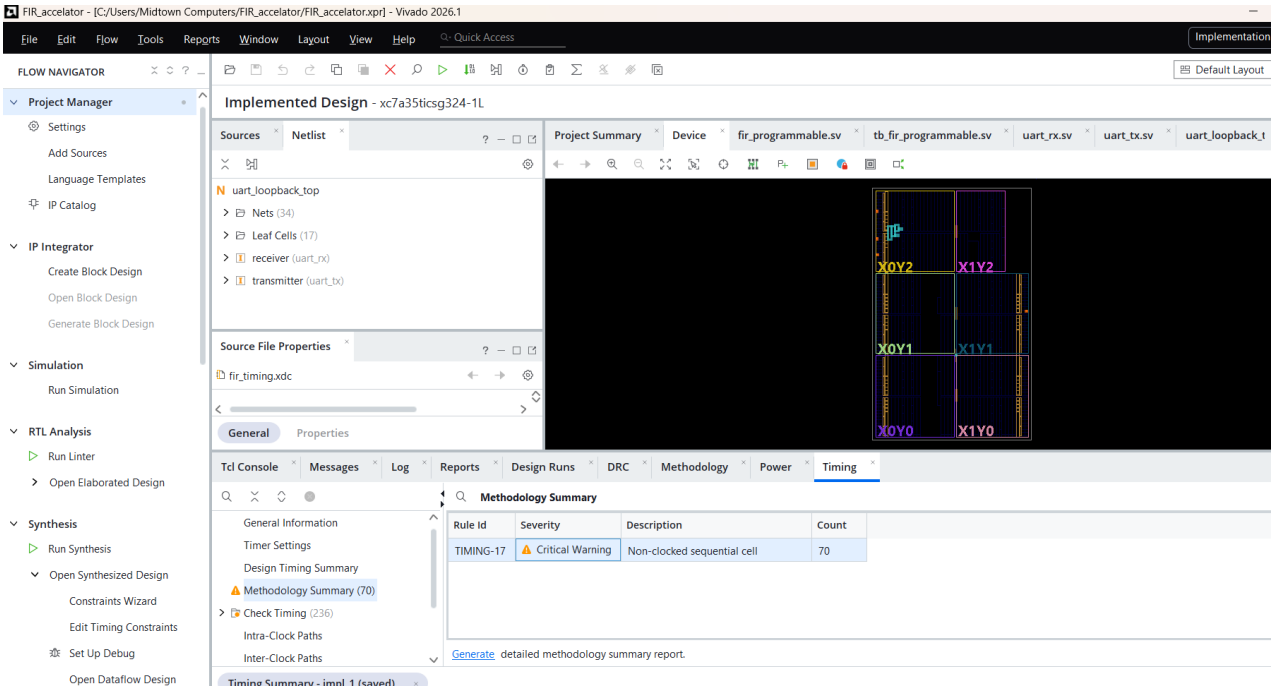
Port / role	Arty Artix-7 pin
clk, 100 MHz	E3
uart_rx_pin, PC -> FPGA	A9
uart_tx_pin, FPGA -> PC	D10
reset, BTN0	D9

The USB cable powers the board, allows Vivado/JTAG to program the SRAM configuration, and exposes a virtual COM5 UART. UART follows 8N1 at 115200. UART RX uses a two-FF synchronizer and a start/data/stop FSM; UART TX uses a corresponding transmit FSM. The loopback initially echoed ASCII A (65).

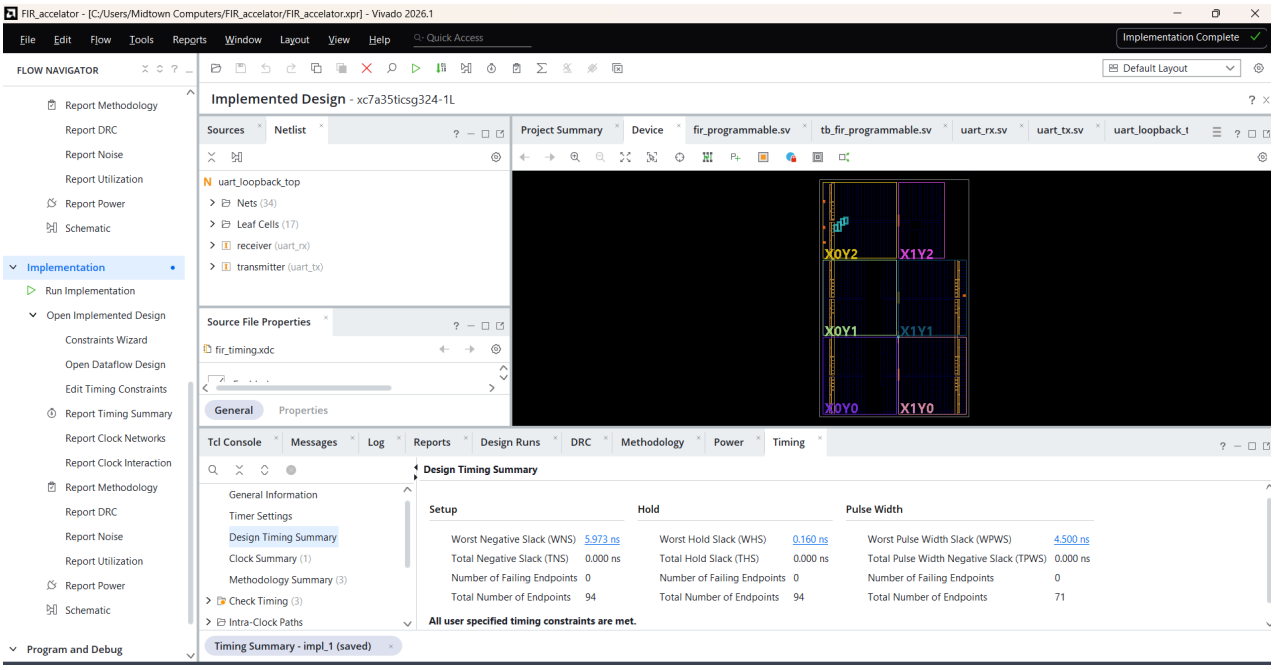
The integration top decodes command S followed by one signed sample byte, waits for output_valid, then sends the result byte; command C is followed by four signed coefficient bytes and returns ASCII K acknowledgement. Since it waits for each transmission to finish, the demonstrated end-to-end design is request-response rather than continuous buffered streaming.

Constraint debugging

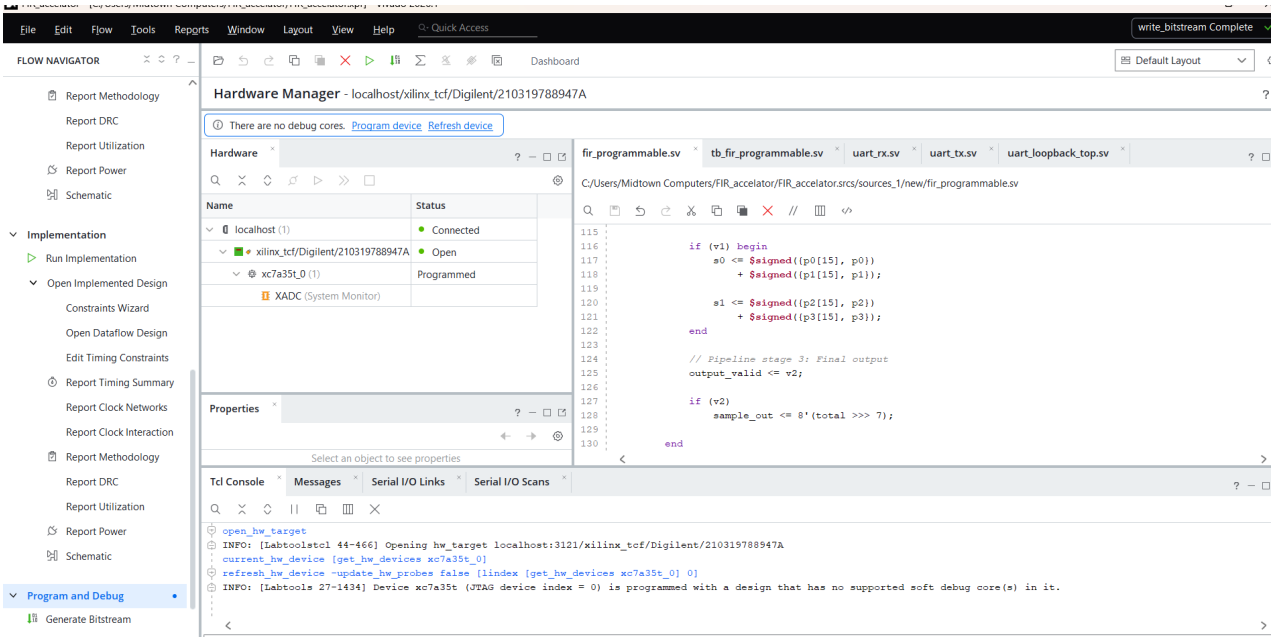
A first UART implementation reported WNS = inf, zero matched clocks and TIMING-17 (70 non-clocked sequential-cell critical warnings). The clock creation constraint had been omitted. Adding create_clock -period 10.000 -name sys_clk [get_ports clk], then rerunning implementation produced a finite +5.973 ns WNS, 0 failing endpoints, and all user-specified timing constraints met. This is a meaningful engineering debugging example.



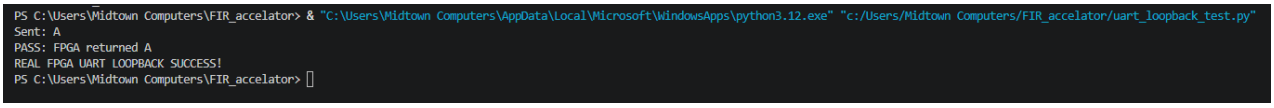
Evidence: TIMING-17 warnings before the clock constraint correction. | Original: image(20261028-155842).png



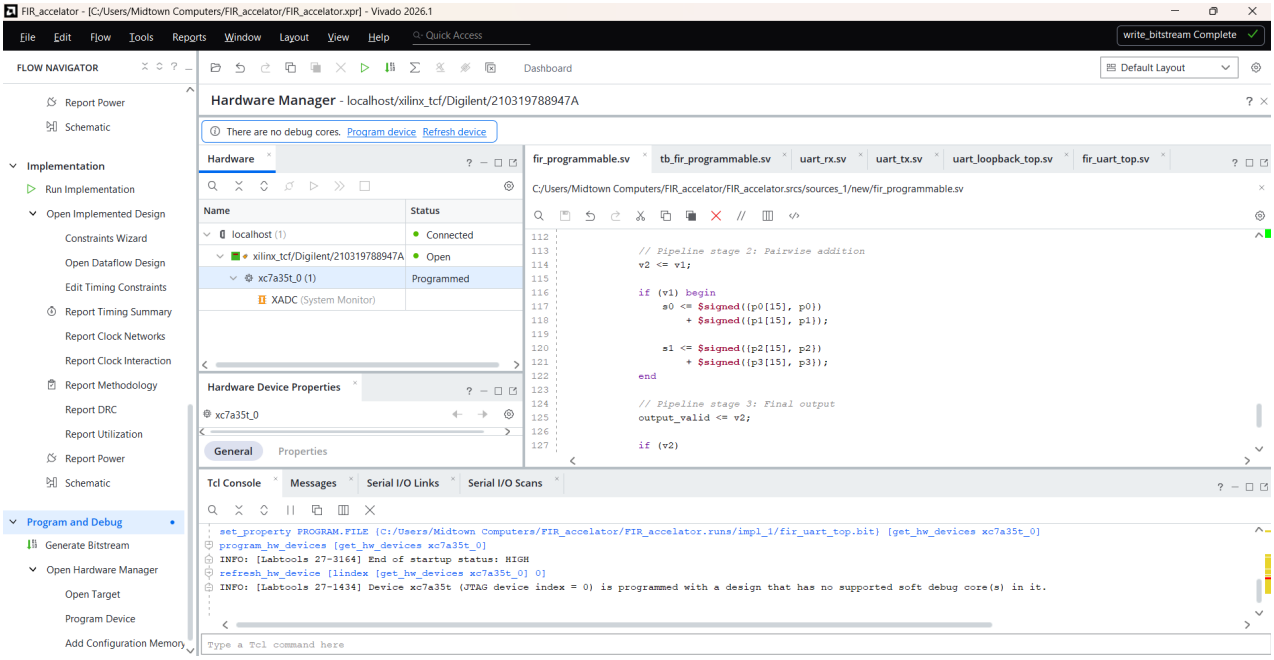
Evidence: UART loopback post-route WNS +5.973 ns after constraint repair. | Original: image(20261028-160335).png



Evidence: Artix-7 device programmed via Vivado Hardware Manager. | Original: image(20261028-160541).png



Evidence: physical UART loopback Python test returns A and passes. | Original: image(20261028-160809).png



Evidence: FIR UART integration top bitstream programmed successfully. | Original: image(20261028-162719).png

8. Dashboard and interactive demonstration

The final Python dashboard allows the user to select COM5, connect/disconnect, enter signed samples, load four signed Q1.7 coefficients, choose preset smoothing/average coefficients, compare hardware outputs against an independent calculation, view current three-sample history, plot input and output traces, and export data to CSV.

The live display screenshot shows 13 samples, 13 matches and 0 mismatches. The physical FPGA output and Python expected traces overlap on the graph, whereas the unfiltered input trace varies significantly. This plot demonstrates the filter smoothing effect for the entered data; it is not an oscilloscope capture of FPGA pins or a frequency-response measurement.



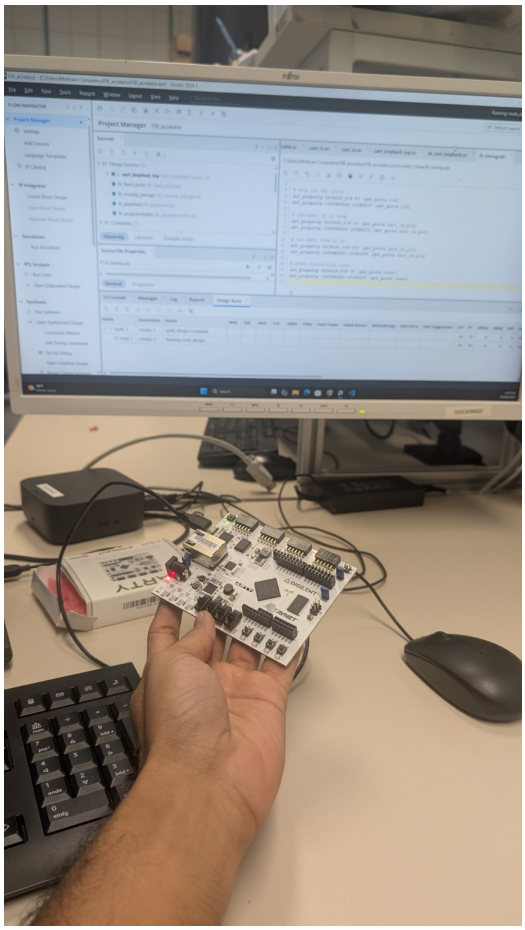
Evidence: dashboard initial screen and manual controls. | Original: image(20261028-171027).png



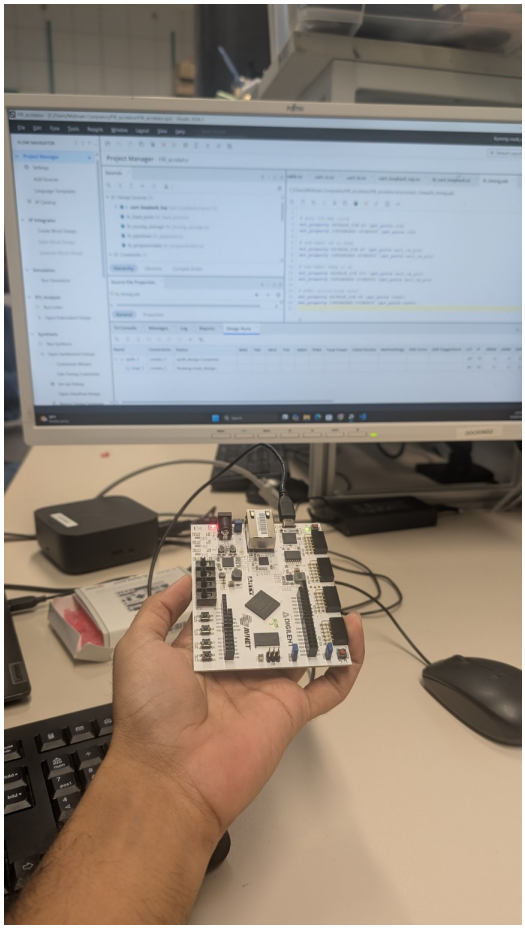
Evidence: dashboard after 13 manually entered samples; curves and counters. | Original: image(20261028-171129).png

9. Photo documentation of physical test setup

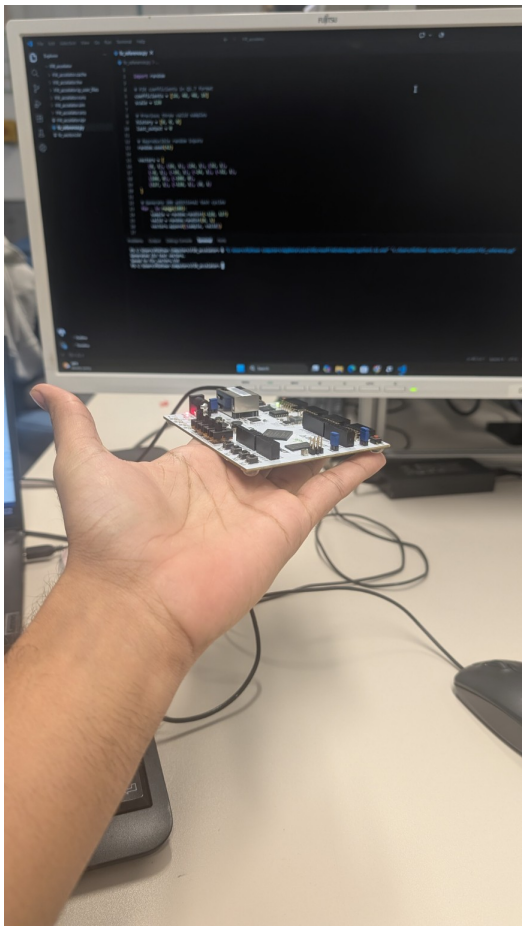
These photographs were supplied as documentation of the connected board, PC, Vivado and VS Code workflow. The screenshots with readable logs above are the quantitative evidence.



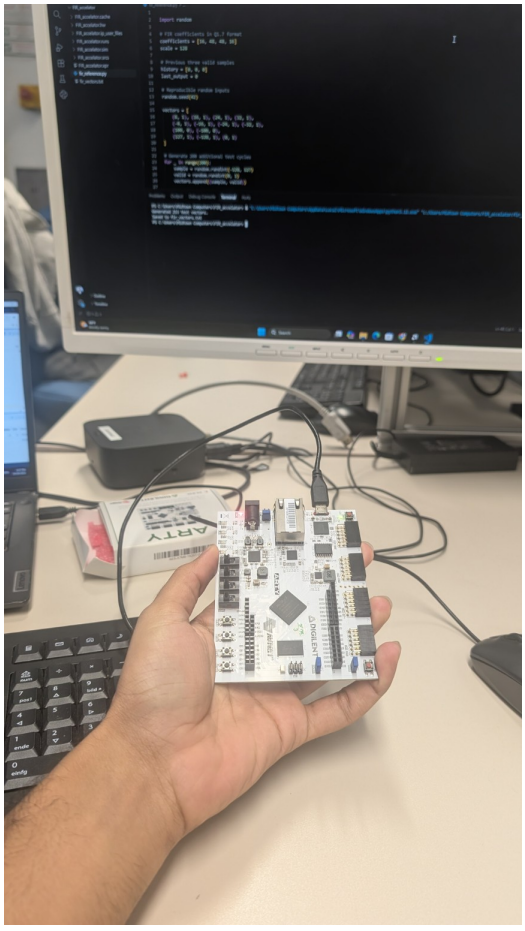
Arty board and development workstation, view 1 | Original: 168253.jpg



Board and Vivado workspace, view 2 | Original: 168252.jpg



Low-angle view of connected Arty and Python development workstation | Original: 168251.jpg



Board connected over USB, front view | Original: 168250.jpg

10. Lessons learned, constraints, and future improvements

Verified: fixed-point math, pipeline logic for test cases, programmable coefficient updates, real JTAG configuration, physical UART receive/transmit, real hardware FIR outputs, interactive dashboard verification.

Not yet claimed: rigorous arbitrary-coefficient overflow behavior; rounded/saturated output; packet error detection; continuous streaming throughput; measured power consumption; instrumented on-board latency / fMAX; fully generalized tap count and width; exhaustive property coverage. These are future enhancements and should not be implied by passing the current tests.

Immediate portfolio release: retain source RTL, all testbenches, Python reference, UART controller, board XDC, terminal results, dashboard code, screenshots, and documented build instructions. Store Vivado generated runs separately or omit them from version control to keep the repository manageable.

Possible next iteration: saturating output with defined rounding; parameterized tap count and coefficient widths; framed UART transactions with acknowledgement and errors; burst transfer with FIFO; ILA-based cycle measurements; Python spectral analysis and comparison of measured filter impulse/frequency response.

11. Reproducibility guide

Stage	Reproduction action
FPGA build	Set fir_uart_top as Design Top; use board pin XDC and 10 ns create_clock
RTL verification	Run tb_fir_python and tb_fir_programmable (keep standalone top selection distinct)
UART verification	Run tb_uart_loopback; send A and confirm echoed A
Physical configuration	Generate fir_uart_top.bit; program xc7a35t_0 using Hardware Manager
Host connection	Connect USB; use available COM port (observed COM5), 115200 8N1
Manual FPGA state reset	Press/release BTN0 before a fresh zero-history test
Run dashboard	Install pyserial and matplotlib, run fir_dashboard.py, connect COM5
Evidence capture	Save screenshots of PASS/FAIL counters and CSV export

12. Image evidence register

The companion ZIP preserves **38** project images (all available screenshots with a 20261028 timestamp and the four supplied board photographs) under their original filenames. The report embeds representative evidence throughout; the ZIP provides the complete chronological visual record.

Image register — continued

01. image(20261028-131107).png

02. image(20261028-132750).png

03. image(20261028-132945).png

04. image(20261028-133014).png

05. image(20261028-133215).png

06. image(20261028-133258).png

07. image(20261028-133443).png

08. image(20261028-141700).png

09. image(20261028-142457).png

10. image(20261028-142526).png
11. image(20261028-142551).png
12. image(20261028-143311).png

Image register — continued

13. image(20261028-144232).png
14. image(20261028-144249).png
15. image(20261028-151048).png
16. image(20261028-151325).png
17. image(20261028-151426).png
18. image(20261028-151717).png
19. image(20261028-153021).png
20. image(20261028-153231).png
21. image(20261028-153503).png
22. image(20261028-153522).png
23. image(20261028-154533).png
24. image(20261028-155109).png

Image register — continued

25. image(20261028-155826).png
26. image(20261028-155842).png
27. image(20261028-160335).png
28. image(20261028-160541).png
29. image(20261028-160809).png
30. image(20261028-162719).png
31. image(20261028-162930).png
32. image(20261028-163234).png
33. image(20261028-171027).png
34. image(20261028-171129).png
35. 168252.jpg
36. 168253.jpg

Image register — continued

37. 168251.jpg
38. 168250.jpg

Conclusion: This is a credible, hands-on FPGA + Python hardware-in-the-loop demonstration, supported by simulation checks, synthesis/place-and-route reports, direct physical UART responses, and dashboard evidence. Report comparisons explicitly separate synthesis timing from post-route measurements and avoid unsupported performance claims.